

Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour



**UNIVERSITY OF
CAMBRIDGE**
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance
Practical Workshop 3	Initialisation and regularisation
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 4	Convolutional neural networks
Practical Workshop 4	Scikit Learn
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

Practical Workshop 1

What we'll do today

1

Linear Regression

Fit $y = \beta_0 + \beta_1 x$ to noisy data

2

Neural Networks

Stack two linear layers with a non-linearity.

Simple Linear Regression

Linear regression is the smallest possible model.

We consider the following model:

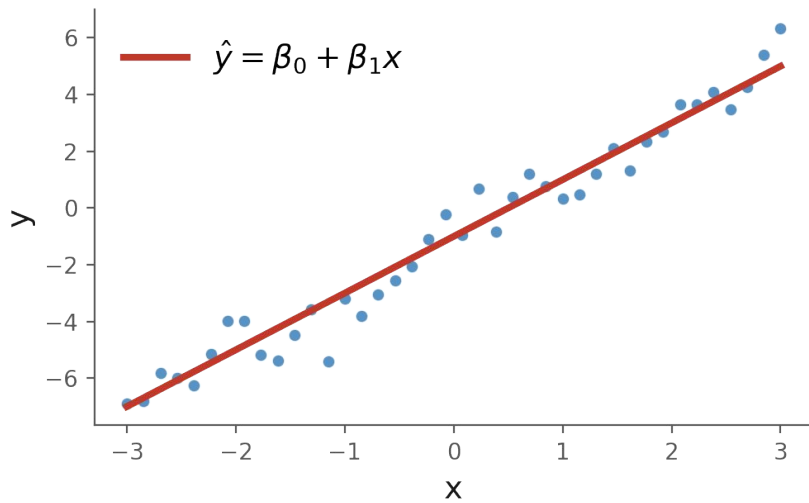
$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

β_0 — the intercept

β_1 — the slope

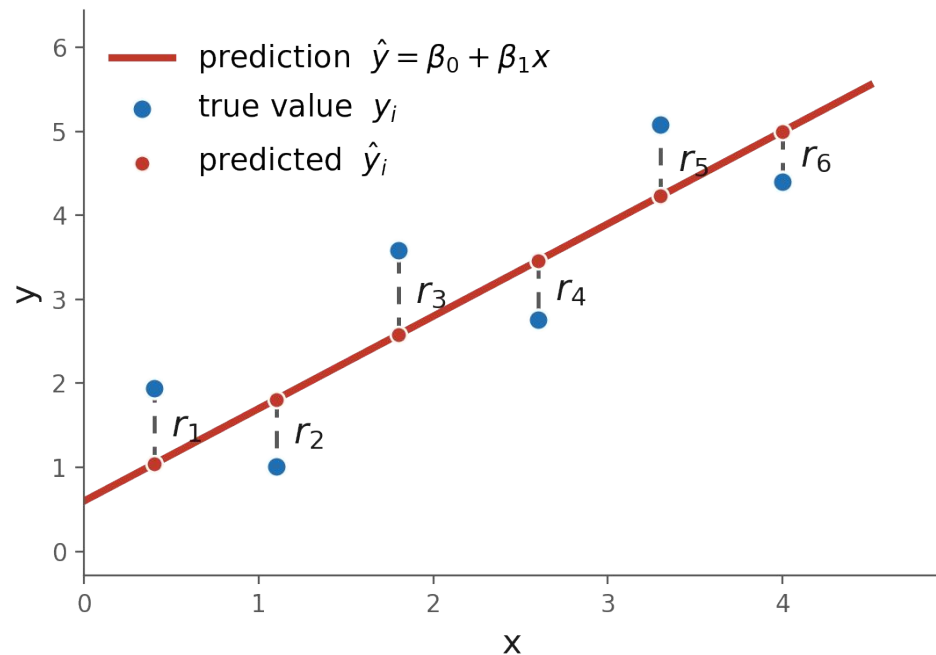
ε_i — Gaussian noise with variance σ^2

We aim to fit a line through the observed data by estimating the parameters β_0, β_1 .



Residual Sum of Squares

Each prediction misses the ground truth by a *residual*. We want those residuals to be small.



We define the residual as:

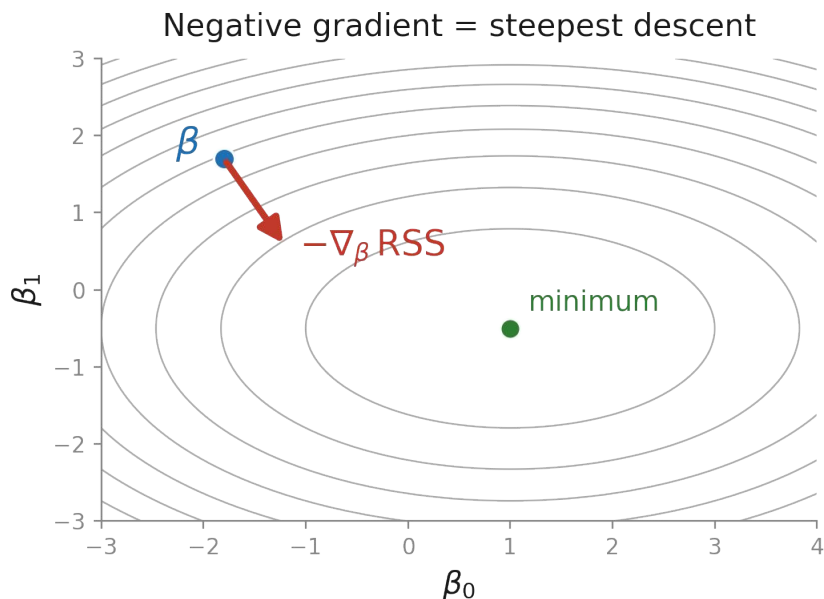
$$r_i = y_i - (\beta_0 + \beta_1 x_i)$$

To compute the aggregate error across all samples, we compute the residual sum of squares:

$$RSS = \sum_{i=1}^N (y_i - (\beta_0 + \beta_1 x_i))^2$$

Goal: $\hat{\beta}_0, \hat{\beta}_1 = \arg \min_{\beta_0, \beta_1} RSS$

Gradient Descent



The gradient of the loss metric with respect to the parameters ($\nabla_{\beta} \text{RSS}$) always points in the direction the loss increases fastest.

Hence, we should aim to update parameters in the direction of $-\nabla_{\beta} \text{RSS}$.

Idea: Take a small step in the direction of $-\nabla_{\beta} \text{RSS}$ such that the loss is guaranteed not to rise:

$$\text{RSS}(\beta - \eta \nabla_{\beta} \text{RSS}) \leq \text{RSS}(\beta)$$

Gradient Descent

$$\beta \leftarrow \beta - \eta \nabla_{\beta} \text{RSS}$$

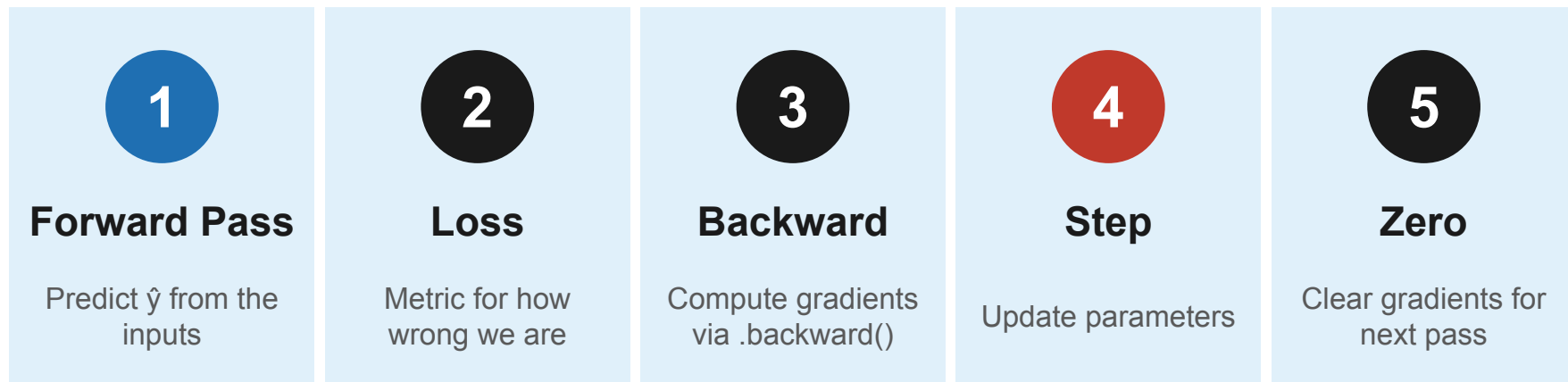
$$\beta_0 \leftarrow \beta_0 - \eta \frac{\partial \text{RSS}}{\partial \beta_0}$$

$$\beta_1 \leftarrow \beta_1 - \eta \frac{\partial \text{RSS}}{\partial \beta_1}$$

η is called the learning rate, and defines how big a step we take in parameter space. If it's too large, we don't guarantee the loss reduces. If it's too small, learning is slow.

Training Loop

Every neural network is trained using the following five steps at its core.

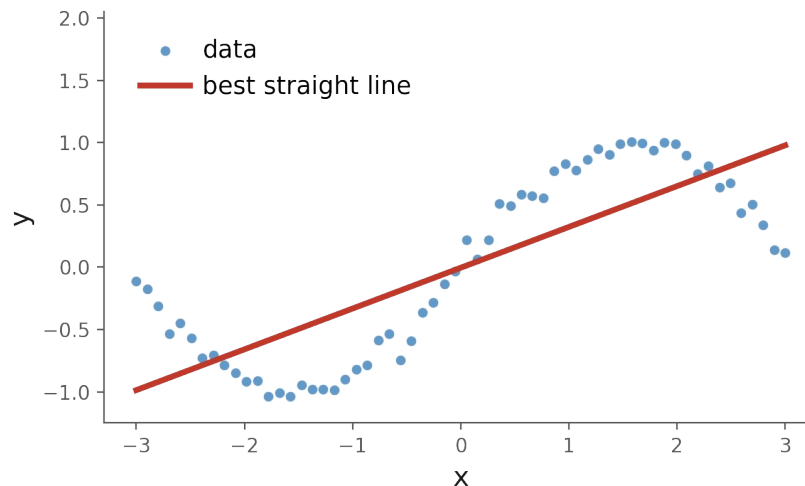


The above is called an *epoch*, and we repeat this until the loss reaches a desired level.

Time for some code...

Non-Linear Relationships

Consider the following dataset:



Stacking multiple linear layers can only ever produce a linear relationship.

For two linear layers:

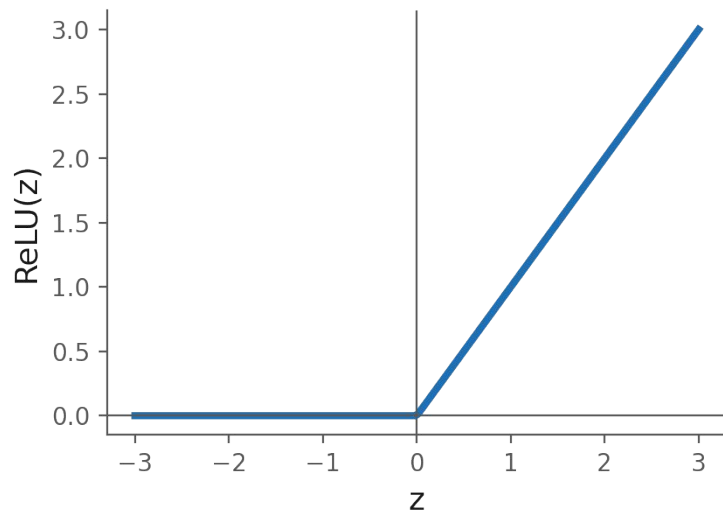
$$(xW_1)W_2 = x(W_1W_2) = xW'$$

This extends for any number of linear layers.

Solution: since most real world data is rarely linear, we insert a deliberate non-linearity to create an inductive bias toward richer functions.

Rectified Linear Unit (ReLU)

The simplest non-linear function is the ReLU activation function.

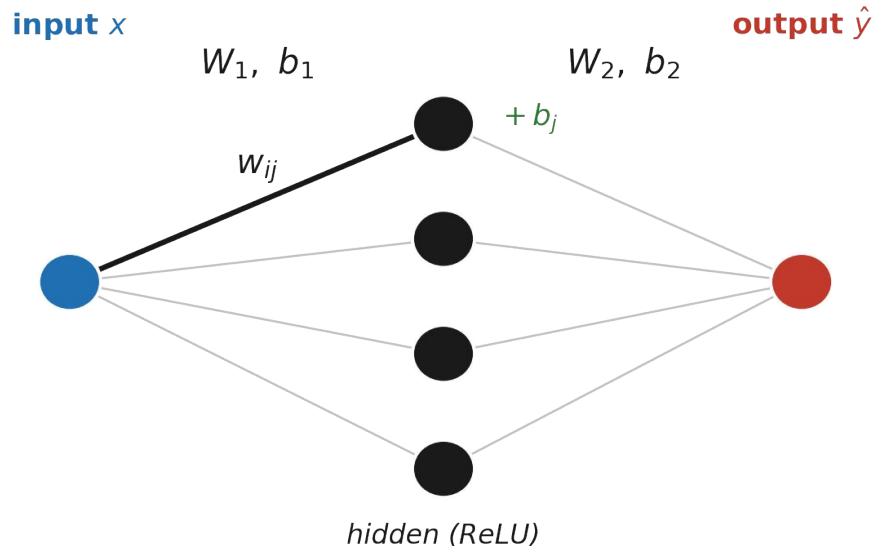


$$\text{ReLU}(z) = \max(0, z)$$

1. Keeps positive values, and zeroes the rest.
2. Cheap to compute, gradients flow nicely.

Two-Layer Neural Network

Let's consider two linear layers, with a ReLU non-linearity.



Each line is a weight
Each node adds a bias

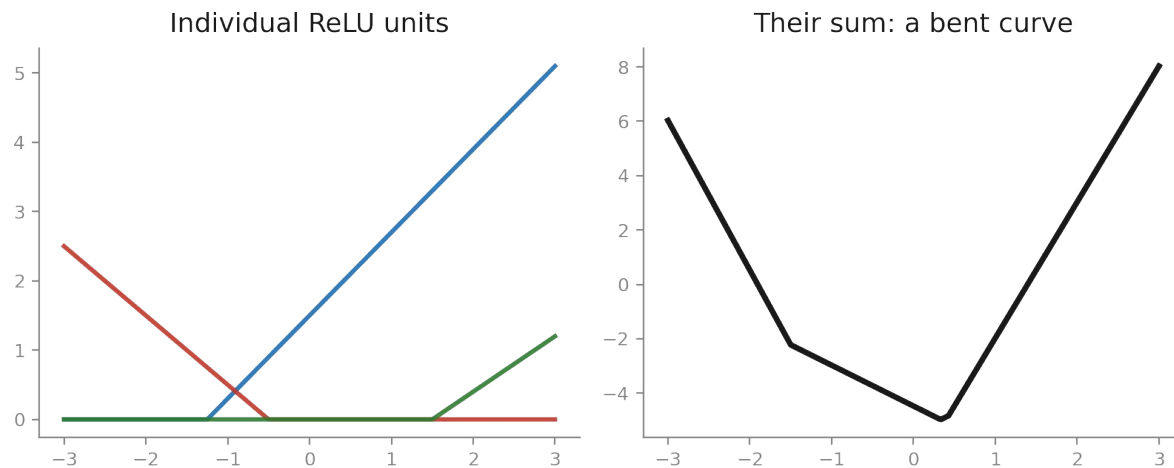
$$h = \text{ReLU}(\overbrace{xW_1 + b_1}^{\text{pre-activation}})$$

$$\hat{y} = h W_2 + b_2$$

Four parameter tensors: W_1, b_1, W_2, b_2

The five-step training loop is used to update all parameters simultaneously.

ReLU Networks



1. Each hidden unit (neuron, node) in a hidden layer introduces a ReLU hinge
2. The output layer combines them into one. This is why the output layer is not passed through a non-linearity.

Universal Approximation Theorem

Networks with one hidden layer of ReLU units are universal function approximators.

Recap

1 **A model is just parameters.**

A line has two (β_0 , β_1), while a neural network will have many more.

2 **Training involves making our errors small.**

Measure error, update parameters by following the gradient downhill, and repeat.

3 **Non-linearity improves expressiveness**

Adding ReLU units between linear layers is what allows linear layers to capture non-linear behaviours.

Questions for you!

1. How are you finding the pace of the lectures and the workshop?
2. How are you finding the difficulty of the lectures and the workshop?
3. Are there any changes that you'd think would make the workshop sessions better?